CAMBRIDGE IGCSE & O LEVEL COMPUTER SCIENCE (0478 / 2210)

Hodder Education Second Edition · Chapter 1: Data Representation
SECTION 1.1: NUMBER SYSTEMS — COMPLETE QUESTION BANK & SOLUTIONS

📖 Reference: Cambridge IGCSE and O Level Computer Science Second Edition (David Watson & Helen Williams) — Chapter 1.1 (Pages 2–24 & 38–44) and 2023–2024 Past Paper Questions.

1. Binary to Denary Conversion (Activity 1.1)

📌 Textbook Page 4 (Activity 1.1)

Convert the following 8-bit binary numbers into denary:

#	Question / Value	Step-by-Step Solution & Answer
a	**00110011**	32 + 16 + 2 + 1 = 51
b	**01111111**	64 + 32 + 16 + 8 + 4 + 2 + 1 = 127
c	**10011001**	128 + 16 + 8 + 1 = 153
d	**01110100**	64 + 32 + 16 + 4 = 116
e	**11111111**	128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255
f	**00001111**	8 + 4 + 2 + 1 = 15
g	**10001111**	128 + 8 + 4 + 2 + 1 = 143
h	**10110011**	128 + 32 + 16 + 2 + 1 = 179
i	**01110000**	64 + 32 + 16 = 112
j	**11101110**	128 + 64 + 32 + 8 + 4 + 2 = 238

2. Denary to Binary Conversion (Activity 1.2)

📌 Textbook Page 7 (Activity 1.2)

Convert the following denary numbers into 8-bit binary numbers (using place value subtraction or successive division by 2):

#	Question / Value	Step-by-Step Solution & Answer
a	**41**	32 + 8 + 1 = 00101001
b	**67**	64 + 2 + 1 = 01000011
c	**86**	64 + 16 + 4 + 2 = 01010110
d	**100**	64 + 32 + 4 = 01100100
e	**111**	64 + 32 + 8 + 4 + 2 + 1 = 01101111

f	**127**	64 + 32 + 16 + 8 + 4 + 2 + 1 = 01111111
g	**144**	128 + 16 = 10010000
h	**189**	128 + 32 + 16 + 8 + 4 + 1 = 10111101
i	**200**	128 + 64 + 8 = 11001000
j	**255**	128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 11111111

3. Binary to Hexadecimal Conversion (Activity 1.3)

Textbook Page 9 (Activity 1.3)

Convert the following binary numbers into hexadecimal (split into 4-bit nibbles from right):

#	Question / Value	Step-by-Step Solution & Answer
a	**1100 0011**	1100 = C, 0011 = 3 -> C3
b	**1111 0111**	1111 = F, 0111 = 7 -> F7
c	**0010 0111 1111**	0010 = 2, 0111 = 7, 1111 = F -> 27F
d	**0100 1110 1110**	0100 = 4, 1110 = E, 1110 = E -> 4EE
e	**0001 1110 0001**	0001 = 1, 1110 = E, 0001 = 1 -> 1E1
f	**1000 1001 1110**	1000 = 8, 1001 = 9, 1110 = E -> 89E
g	**0111 0100 1110**	0111 = 7, 0100 = 4, 1110 = E -> 74E

4. Hexadecimal to Binary Conversion (Activity 1.4)

Textbook Page 10 (Activity 1.4)

Convert the following hexadecimal values into binary (expand each hex digit into 4-bit binary):

#	Question / Value	Step-by-Step Solution & Answer
a	**6C**	6 = 0110, C (12) = 1100 -> 01101100
b	**59**	5 = 0101, 9 = 1001 -> 01011001
c	**AA**	A (10) = 1010, A (10) = 1010 -> 10101010
d	**A00**	A = 1010, 0 = 0000, 0 = 0000 -> 101000000000
e	**40E**	4 = 0100, 0 = 0000, E (14) = 1110 -> 010000001110
f	**BA6**	B (11) = 1011, A = 1010, 6 = 0110 -> 101110100110
g	**9CC**	9 = 1001, C = 1100, C = 1100 -> 100111001100

h	**40AA**	4 = 0100, 0 = 0000, A = 1010, A = 1010 -> 0100000010101010
i	**DA47**	D (13) = 1101, A = 1010, 4 = 0100, 7 = 0111 -> 1101101001000111
j	**1AB0**	1 = 0001, A = 1010, B = 1011, 0 = 0000 -> 0001101010110000

5. Hexadecimal to Denary Conversion (Activity 1.5)

Textbook Page 11 (Activity 1.5)

Convert the following hexadecimal values into denary:

#	Question / Value	Step-by-Step Solution & Answer
a	**6B**	(6 * 16) + 11 = 96 + 11 = 107
b	**9C**	(9 * 16) + 12 = 144 + 12 = 156
c	**4A**	(4 * 16) + 10 = 64 + 10 = 74
d	**FF**	(15 * 16) + 15 = 240 + 15 = 255
e	**1FF**	(1 * 256) + (15 * 16) + 15 = 256 + 240 + 15 = 511
f	**A01**	(10 * 256) + (0 * 16) + 1 = 2560 + 1 = 2561
g	**BB4**	(11 * 256) + (11 * 16) + 4 = 2816 + 176 + 4 = 2996
h	**CA8**	(12 * 256) + (10 * 16) + 8 = 3072 + 160 + 8 = 3240
i	**12AE**	(1 * 4096) + (2 * 256) + (10 * 16) + 14 = 4096 + 512 + 160 + 14 = 4782
j	**AD89**	(10 * 4096) + (13 * 256) + (8 * 16) + 9 = 40960 + 3328 + 128 + 9 = 44425

6. Denary to Hexadecimal Conversion (Activity 1.6)

Textbook Page 12 (Activity 1.6)

Convert the following denary numbers into hexadecimal:

#	Question / Value	Step-by-Step Solution & Answer
a	**98**	98 / 16 = 6 remainder 2 -> 62
b	**227**	227 / 16 = 14 (E) remainder 3 -> E3
c	**490**	490 / 16 = 30 rem 10 (A); 30 / 16 = 1 rem 14 (E) -> 1EA

d	**511**	511 / 16 = 31 rem 15 (F); 31 / 16 = 1 rem 15 (F) -> 1FF
e	**826**	826 / 16 = 51 rem 10 (A); 51 / 16 = 3 rem 3 -> 33A
f	**1000**	1000 / 16 = 62 rem 8; 62 / 16 = 3 rem 14 (E) -> 3E8
g	**2634**	2634 = (10 * 256) + (4 * 16) + 10 -> A4A
h	**3743**	3743 = (14 * 256) + (9 * 16) + 15 -> E9F
i	**4007**	4007 = (15 * 256) + (10 * 16) + 7 -> FA7
j	**5000**	5000 = (1 * 4096) + (3 * 256) + (8 * 16) + 8 -> 1388

7. Binary Addition & Carry Bits (Activity 1.8 & 1.9)

Textbook Page 16 (Activity 1.8 & 1.9)

Perform the following 8-bit binary additions:

#	Question / Value	Step-by-Step Solution & Answer
a	**00011101 + 01100110**	(29 + 102) = 010000011 = 10000011 (131)
b	**00100111 + 00111111**	(39 + 63) = 01100110 (102)
c	**00101110 + 01001101**	(46 + 77) = 01111011 (123)
d	**01110111 + 00111111**	(119 + 63) = 10110110 (182)
e	**00111100 + 00110011**	(60 + 51) = 01101111 (111)
f	**00111100 + 01111011**	(60 + 123) = 10110111 (183)
g	**00111111 + 00111111**	(63 + 63) = 01111110 (126)
h	**00110001 + 00111111**	(49 + 63) = 01110000 (112)
i	**01111111 + 01111111**	(127 + 127) = 11111110 (254)
j	**10100010 + 00111011**	(162 + 59) = 11011101 (221)

8. Binary Addition Overflow Errors (Activity 1.10)

Textbook Page 17 (Activity 1.10)

Convert the following pairs of denary numbers into 8-bit binary and add them. Explain why an overflow error occurs:

#	Question / Value	Step-by-Step Solution & Answer
a	**89 + 175**	89 = 01011001, 175 = 10101111 -> Sum = 264 -> 1

		00001000. Overflow error: Result (264) exceeds the maximum 8-bit value of 255; requires a 9th bit.
b	**168 + 99**	168 = 10101000, 99 = 01100011 -> Sum = 267 -> 1 00001011. Overflow error: Carry out from bit 7 (MSB) cannot be stored in an 8-bit register.
c	**88 + 215**	88 = 01011000, 215 = 11010111 -> Sum = 303 -> 1 00101111. Overflow error: Result exceeds 255.

9. Logical Binary Shifts (Activity 1.11)

Textbook Page 18-19 (Activity 1.11)

Logical shifts, multiplication/division, and precision loss:

#	Question / Value	Step-by-Step Solution & Answer
a	**Write 24 as an 8-bit register and shift 2 places left**	24 = 00011000 -> Shift 2 places left: 01100000 = 64 + 32 = 96 (24 * 4 = 96).
b	**Write 200 as an 8-bit register and shift 1 place right**	200 = 11001000 -> Shift 1 place right: 01100100 = 64 + 32 + 4 = 100 (200 / 2 = 100).
c	**Convert 75 to an 8-bit binary number and shift 3 places left**	75 = 01001011 -> Shift 3 places left: 01011000 = 88. Error: A 1-bit was shifted out of MSB (overflow); true value 75 * 8 = 600 cannot fit in 8 bits.
d	**Convert 19 to an 8-bit binary number and shift 1 place right**	19 = 00010011 -> Shift 1 place right: 00001001 = 9. Precision loss: The least significant 1-bit was shifted out (19 / 2 = 9.5, decimal part lost).

10. Two's Complement Representation — Positive Numbers (Activity 1.12)

Textbook Page 22 (Activity 1.12)

Convert the following positive denary numbers into 8-bit two's complement binary format (MSB is 0):

#	Question / Value	Step-by-Step Solution & Answer
a	**+39**	00100111
b	**+66**	01000010
c	**+88**	01011000
d	**+102**	01100110
e	**+111**	01101111

f	**+125**	01111101
g	**+77**	01001101
h	**+20**	00010100
i	**+49**	00110001
j	**+56**	00111000

11. Two's Complement Representation — Negative Denary to Binary (Activity 1.13)

Textbook Page 24 (Activity 1.13)

Convert the following negative denary numbers into 8-bit two's complement binary format (invert bits and add 1):

#	Question / Value	Step-by-Step Solution & Answer
a	**-18**	+18 = 00010010 -> Invert: 11101101 -> Add 1: 11101110 (-128 + 64 + 32 + 8 + 4 + 2 = -18)
b	**-31**	+31 = 00011111 -> Invert: 11100000 -> Add 1: 11100001 (-128 + 64 + 32 + 1 = -31)
c	**-47**	+47 = 00101111 -> Invert: 11010000 -> Add 1: 11010001 (-128 + 64 + 16 + 1 = -47)
d	**-63**	+63 = 00111111 -> Invert: 11000000 -> Add 1: 11000001 (-128 + 64 + 1 = -63)
e	**-88**	+88 = 01011000 -> Invert: 10100111 -> Add 1: 10101000 (-128 + 32 + 8 = -88)
f	**-92**	+92 = 01011100 -> Invert: 10100011 -> Add 1: 10100100 (-128 + 32 + 4 = -92)
g	**-100**	+100 = 01100100 -> Invert: 10011011 -> Add 1: 10011100 (-128 + 16 + 8 + 4 = -100)
h	**-1**	+1 = 00000001 -> Invert: 11111110 -> Add 1: 11111111 (-128 + 127 = -1)
i	**-16**	+16 = 00010000 -> Invert: 11101111 -> Add 1: 11110000 (-128 + 64 + 32 + 16 = -16)
j	**-127**	+127 = 01111111 -> Invert: 10000000 -> Add 1: 10000001 (-128 + 1 = -127)

12. Two's Complement Representation — Negative Binary to Denary (Activity 1.14)

Textbook Page 24 (Activity 1.14)

Convert the following 8-bit negative two's complement binary numbers into denary (using -128 place weighting):

#	Question / Value	Step-by-Step Solution & Answer
a	**11001101**	-128 + 64 + 8 + 4 + 1 = -51
b	**10111110**	-128 + 32 + 16 + 8 + 4 + 2 = -66
c	**11101111**	-128 + 64 + 32 + 8 + 4 + 2 + 1 = -17
d	**10000111**	-128 + 4 + 2 + 1 = -121
e	**10100000**	-128 + 32 = -96
f	**11111001**	-128 + 64 + 32 + 16 + 8 + 1 = -7
g	**10101111**	-128 + 32 + 8 + 4 + 2 + 1 = -81
h	**11111111**	-128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -1
i	**10000001**	-128 + 1 = -127
j	**11110110**	-128 + 64 + 32 + 16 + 4 + 2 = -10

13. Binary Coded Decimal (BCD) — Extension Topic 1

Textbook Page 38 (Extension Topic 1)

Convert denary digits into 4-bit BCD codes and vice-versa (used in clock displays & calculators):

#	Question / Value	Step-by-Step Solution & Answer
a	**Convert denary 271 into BCD**	2 = 0010, 7 = 0111, 1 = 0001 -> 0010 0111 0001
b	**Convert denary 5006 into BCD**	5 = 0101, 0 = 0000, 0 = 0000, 6 = 0110 -> 0101 0000 0000 0110
c	**Convert denary 7990 into BCD**	7 = 0111, 9 = 1001, 9 = 1001, 0 = 0000 -> 0111 1001 1001 0000
d	**Convert BCD 1001 0011 0111 into denary**	1001 = 9, 0011 = 3, 0111 = 7 -> 937
e	**Convert BCD 0111 0111 0110 0010 into denary**	0111 = 7, 0111 = 7, 0110 = 6, 0010 = 2 -> 7762

14. Binary Subtraction using Two's Complement Addition — Extension Topic 2

Textbook Page 38-39 (Extension Topic 2)

Carry out subtraction A - B by converting B into negative two's complement and adding (A + (-B)):

#	Question / Value	Step-by-Step Solution & Answer
a	**95 - 68**	95 = 01011111; 68 = 01000100 -> -68 = 10111100. Sum = 01011111 + 10111100 = 1 00011011. Discard 9th carry bit -> 00011011 = +27 (correct: 95 - 68 = 27).
b	**49 - 80**	49 = 00110001; 80 = 01010000 -> -80 = 10110000. Sum = 00110001 + 10110000 = 11100001 (-128 + 64 + 32 + 1 = -31; correct: 49 - 80 = -31).
c	**01100011 - 00110000 (99 - 48)**	00110000 -> Two's comp: 11010000. Sum = 01100011 + 11010000 = 1 00110011 -> 00110011 = +51 (correct: 99 - 48 = 51).
d	**01111111 - 01011010 (127 - 90)**	01011010 -> Two's comp: 10100110. Sum = 01111111 + 10100110 = 1 00100101 -> 00100101 = +37 (correct: 127 - 90 = 37).

15. Exam-Style Question: Digital Stopwatch Registers

Textbook Page 43 (Exam-Style Question 7 / Cambridge 0478/12/M/J/18 Q3)

A stopwatch uses three 8-bit registers to store Hours, Minutes, and Seconds (HH : MM : SS):

#	Question / Value	Step-by-Step Solution & Answer
a	**Stopwatch displays 02 : 31 : 58.** **Write the 8-bit binary registers.**	Hours (2) = 00000010 Minutes (31) = 00011111 Seconds (58) = 00111010
b	**When stopped later, registers show:** **Hours: 00000101** **Minutes: 00011010** **Seconds: 00110111** **Write the denary time.**	Hours: 00000101 = 4 + 1 = 05 Minutes: 00011010 = 16 + 8 + 2 = 26 Seconds: 00110111 = 32 + 16 + 4 + 2 + 1 = 55 Time Display: 05 : 26 : 55